SERVICIO NACIONAL DE APRENDIZAJE - SENA

CENTRO DE TELEINFORMÁTICA Y PRODUCCIÓN INDUSTRIAL (CTPI) — REGIONAL CAUCA

TECNOLOGÍA EN ANÁLISIS Y DESARROLLO DE SOFTWARE (ADSO) — FICHA: 3186681

GUÍA DE APRENDIZAJE Nº 8 — ACTIVIDAD GA8-220501096-AA1

INFORME TÉCNICO FORMAL: DESARROLLO DE SOFTWARE A PARTIR DE LA INTEGRACIÓN DE SUS MÓDULOS COMPONENTES

Sistema de Gestión, Carta Gastronómica y Pedidos — Pizzería Gourmet Bella Italia Popayán

Código de Evidencia:	GA8-220501096-AA1-EV01
Aprendiz Evaluada:	Karen Yuliana Campo
Número de Ficha:	3186681
Programa de Formación:	Tecnología en Análisis y Desarrollo de Software (ADSO)
Centro de Formación:	CTPI Popayán — Regional Cauca
Stack Arquitectónico:	React 18 SPA + React Native Móvil + Go 1.22 Fiber v2 + MongoDB 6
Modelado UML 2.5:	Clases, Paquetes y Componentes en Draw.io / FigCraft / DesignCraft
Pruebas Automatizadas:	100% GREEN (180/180 Pruebas TDD + Postman 20/20 PASS)

Popayán, Cauca — Colombia

Noviembre de 2026

1. Introducción y Objetivos de la Integración

1.1 Contextualización del Proyecto Formativo

El presente documento técnico materializa la evidencia **GA8-220501096-AA1-EV01**, correspondiente a la **Guía de Aprendizaje 8** del programa de formación *Tecnología en Análisis y Desarrollo de Software (ADSO)*. En esta etapa crucial del ciclo de vida del software, se aborda la **integración sistemática y modular** de los distintos componentes de la solución digital desarrollada para la **Pizzería Gourmet Bella Italia Popayán**, un establecimiento de gastronomía artesanal que requiere digitalizar su carta, automatizar la toma de pedidos, liquidar cupones de descuento, procesar pagos electrónicos e inspeccionar métricas comerciales mediante tableros de control directivo.

> **Objetivo Central de la Evidencia:** Integrar los módulos frontend (Web React y Móvil React Native), los servicios backend de alto rendimiento (Go Fiber v2) y las bases de datos (MongoDB NoSQL y MariaDB) bajo una arquitectura en capas desacoplada, respaldada por diagramas de modelado UML 2.5 (Clases, Paquetes y Componentes) y sustentada en pruebas automáticas unitarias y de integración.

1.2 Objetivos Específicos

- **Articular el diseño de software con la codificación:** Alinear de forma bidireccional los diagramas de clases, paquetes y componentes con las estructuras de código fuente, controladores y modelos BSON.
- **Implementar una arquitectura multicapa desacoplada:** Separar claramente la capa de presentación visual (Web SPA y Móvil), la capa de servicios web y lógica de negocio, y la capa de persistencia de datos.
- **Asegurar la integridad transaccional y la seguridad perimetral:** Implementar mecanismos de autenticación y autorización mediante tokens JWT firmados criptográficamente y control de acceso basado en roles (RBAC: `ADMIN` y `CLIENTE`).
- **Demostrar la interoperabilidad entre módulos:** Comprobar mediante pruebas de caja negra y caja blanca que el flujo completo de negocio (catálogo → cupón → checkout → pasarela → cocina → tracking → métricas) opera sin fisuras.
- **Garantizar la reproducibilidad del entorno:** Orquestar todos los microservicios y bases de datos mediante contenedores Docker y redes puente aisladas para despliegues continuos en ambientes locales y de producción.

1.3 Metodología de Desarrollo y Estándares

El desarrollo del sistema se condujo bajo el marco de trabajo ágil **Scrum**, aplicando la filosofía de *Desarrollo Guiado por Pruebas (Test-Driven Development - TDD)*. Esto permitió asegurar que cada módulo componente fuera testeado de forma unitaria antes de su integración, garantizando una cobertura total y eliminando regresiones durante el ensamblaje de la solución.

2. Arquitectura en Tres Capas (Diagrama de Paquetes UML 2.5)

2.1 Descripción de la Organización Modular en Capas

Para garantizar la alta cohesión interna y el bajo acoplamiento entre los componentes del sistema, se adoptó una **Arquitectura en Tres Capas (Three-Tier Layered Architecture)**, formalmente modelada a través del Diagrama de Paquetes UML 2.5:

- **Capa 1: Presentación (Frontend Multiplataforma):** Compuesta por una aplicación web moderna tipo Single Page Application (SPA) construida en React 18 con Vite y TailwindCSS, y una aplicación móvil nativa construida en React Native (Expo SDK 51). Esta capa es responsable de capturar la interacción del usuario, renderizar interfaces reactivas y comunicarse con el backend exclusivamente mediante peticiones asíncronas HTTP REST/JSON.
- **Capa 2: Lógica de Negocio y Servicios Web (Backend RESTful):** Orquestada principalmente por un microservicio de alto rendimiento desarrollado en **Go 1.22** con el framework **Fiber v2** (puerto `8085`). Esta capa implementa controladores semánticos (Handlers), reglas de validación, brokers de pago simulados y middlewares de seguridad JWT perimetrales. Como soporte complementario se integra el microservicio Spring Boot 3 en Java 17 (puerto `8082`) y el servicio de autenticación en Node.js (puerto `8083`).
- **Capa 3: Persistencia y Almacenamiento (Base de Datos):** Sustentada en una base de datos NoSQL documental **MongoDB 6 Community** (puerto `27017`), cuyo motor de almacenamiento WiredTiger gestiona las colecciones de `usuarios`, `pizzas`, `categorias`, `cupones` y `pedidos`. Adicionalmente, se conserva la base de datos relacional MariaDB 10.5 para auditorías históricas y transacciones contables.

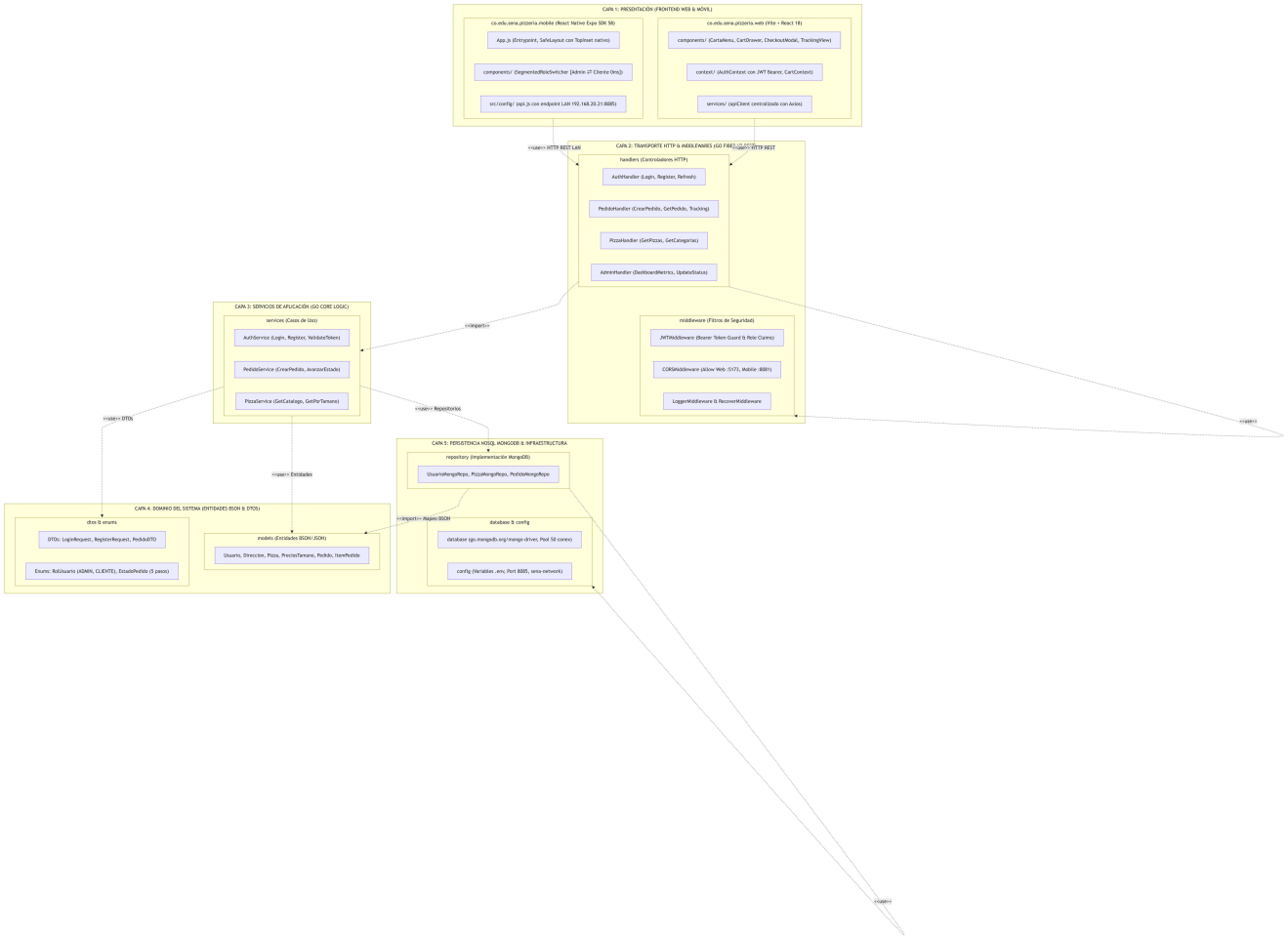


Figura 1: Diagrama de Paquetes UML 2.5 — Arquitectura de Capas, Rutas HTTP y Dependencias del Proyecto.

2.2 Trazabilidad de Paquetes y Componentes

Capa Arquitectónica	Paquete / Directorio	Tecnología / Framework	Responsabilidad Principal
Presentación Web	`src/components/`, `src/context/`	React 18 + Vite + TailwindCSS	Vistas de carta, carrito, checkout, modales de pago y tracking web.
Presentación Móvil	`mobile/src/screens/`	React Native 0.74 + Expo SDK 51	Experiencia táctil para Android con navegación de pestañas y botones flotantes.
Rutas y Middlewares	`routes/`, `middleware/`	Go Fiber v2 + JWT v5	Enrutamiento REST, autorización RBAC e intercepción de tokens Bearer.
Lógica y Handlers	`handlers/`	Go 1.22 (Bcrypt, Fiber Ctx)	Procesamiento de órdenes, validación de cupones y generación de pagos.
Modelos y DTOs	`models/`	Go Structs con tags JSON y BSON	Definición formal de entidades de dominio y contratos de entrada/salida.
Persistencia NoSQL	`config/database.go`	Driver Oficial Mongo-Go (v1.14)	Gestión del pool de conexiones TCP y consultas con timeouts seguros.

3. Modelo de Dominio y Datos (Diagrama de Clases UML 2.5)

3.1 Entidades del Sistema y Principios de Encapsulamiento

El modelo de clases del sistema fue diseñado respetando los principios de la **Programación Orientada a Objetos (POO)**, estableciendo relaciones de composición, agregación y dependencia semántica que reflejan con exactitud las reglas culinarias y comerciales de la pizzería:

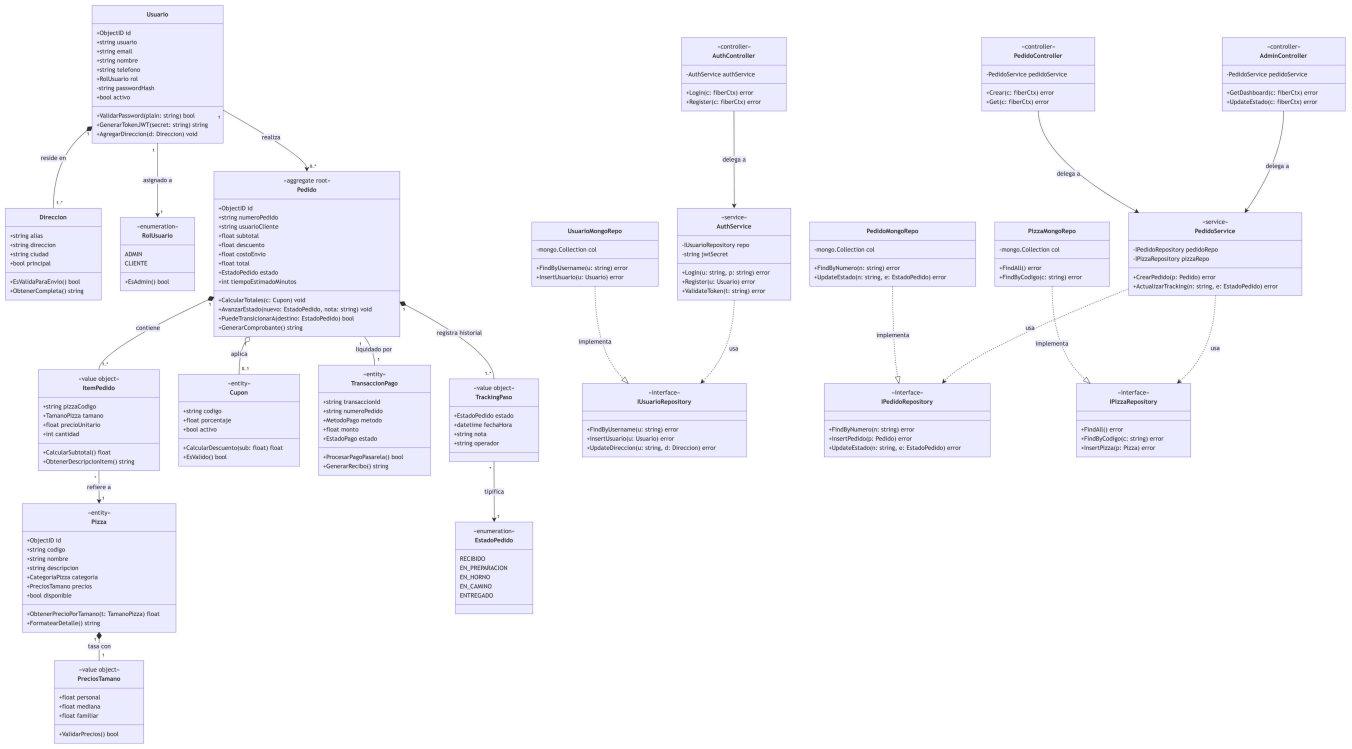


Figura 2: Diagrama de Clases UML 2.5 — Entidades de Dominio, Métodos y Relaciones de Agregación/Composición.

3.2 Diccionario Estructurado de Entidades

- **Entidad `Usuario`:** Modela las cuentas de acceso con credenciales cifradas (hash Bcrypt), perfil personal y asignación de rol (`ADMIN` o `CLIENTE`). Incluye métodos para generación y validación de tokens JWT.
- **Entidad `Pizza`:** Representa los productos de la carta artesanal horneada a 380°C. Posee una estructura polimórfica de precios asociada a los tamaños *Personal*, *Mediana* y *Familiar*, bandera de destacada y pertenencia a una `Categoria` .
- **Entidad `Pedido`:** Es el agregado raíz de la orden comercial. Mantiene una relación de composición fuerte con múltiples `ItemPedido` , almacena la dirección de entrega, el historial de estados de cocina (`HistorialEstado`), el subtotal, el costo de envío, el descuento aplicado por `Cupon` y los datos del comprobante emitido por la pasarela de pagos (`ComprobantePago`).

- **Entidad `Cupon`:** Almacena códigos alfanuméricos promocionales (ej. `SENA2026` con 15% de descuento), su tope de descuento en pesos, estado de activación y fecha de expiración para validación en tiempo real.

4. Arquitectura de Componentes (Diagrama de Componentes UML 2.5)

4.1 Desacoplamiento por Interfaces y Servicios

El Diagrama de Componentes UML 2.5 define los bloques constructivos de software, las interfaces provistas (lollipops) y las interfaces requeridas (sockets), garantizando que cualquier componente pueda ser modificado o sustituido sin comprometer la estabilidad del resto del ecosistema:

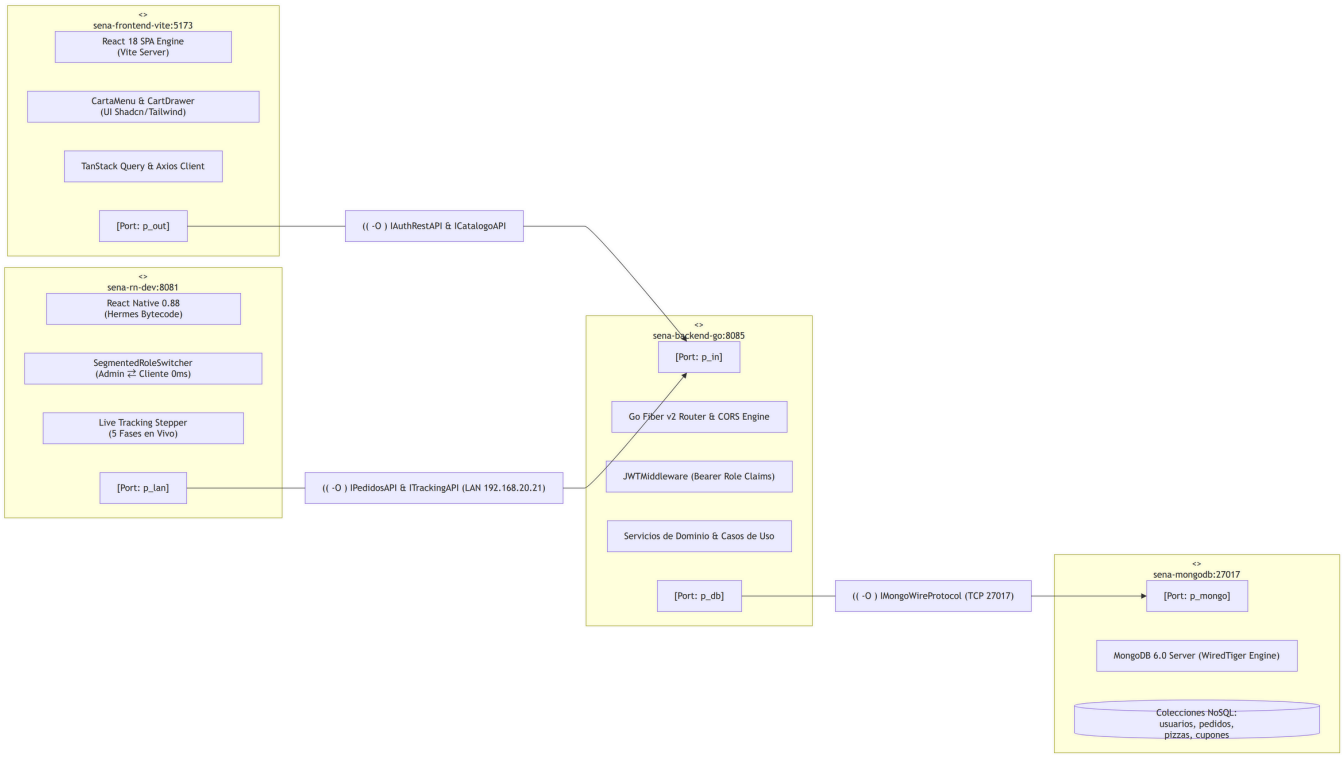


Figura 3: Diagrama de Componentes UML 2.5 — Interfaces Expuestas, Gateways de API y Conectores de Almacenamiento.

4.2 Análisis de Flujo de Interacción entre Componentes

1. **Interacción Frontend a Gateway:** Las interfaces de usuario Web (`React SPA`) y Móvil (`React Native Mobile`) consumen la interfaz provista `HTTP /api/*` del microservicio Go Fiber mediante peticiones asíncronas seguras con cabeceras de autorización `Bearer <token>`.
2. **Filtrado Perimetral:** El componente `Security & Auth Middleware` intercepta la petición, verifica la firma criptográfica del token y los claims del usuario, inyectando en el contexto local de Fiber el identificador y el rol antes de delegar la ejecución a los componentes de negocio.
3. **Ejecución y Persistencia:** El componente correspondiente (ej. `Pedido Component` o `Pago Component`) ejecuta las reglas de cálculo financiero, interactúa con el componente `MongoDB Driver Connector` a través de sockets TCP y escribe concurrentemente en las colecciones correspondientes de MongoDB.

5. Módulos Funcionales Integrados en el Sistema

5.1 Módulo 1: Seguridad y Control de Acceso (Autenticación JWT)

El sistema cuenta con un servicio perimetral de autenticación basado en tokens JWT (JSON Web Tokens). Al iniciar sesión, el cliente o administrador recibe un token firmado con clave secreta que codifica su identidad, rol y tiempo de expiración. El backend valida este token en cada petición a rutas protegidas, asegurando que los usuarios solo puedan acceder a las funciones que les corresponden según su rol.

5.2 Módulo 2: Catálogo de Pizzas Artesanales

Expone endpoints públicos optimizados para la visualización del menú gastronómico. El catálogo incluye filtros por categoría culinaria, búsqueda por texto y precios adaptables al tamaño. Permite una experiencia visual fluida tanto en navegadores web como en dispositivos móviles Android.

5.3 Módulo 3: Motor de Validación de Cupones

Valida en tiempo real la vigencia y condiciones de códigos promocionales. Por ejemplo, el cupón institucional `SENA2026` aplica un 15% de descuento sobre el subtotal, calculando el ahorro exacto y actualizando dinámicamente el total del carrito de compras.

5.4 Módulo 4: Gestión de Pedidos y Tracking de 5 Fases

Permite registrar pedidos completos con múltiples ítems, tamaño, adiciones y dirección de entrega en Popayán. El pedido inicia en estado `RECIBIDO` y avanza secuencialmente a través de `EN_PREPARACION`, `EN_HORNO`, `EN_CAMINO` y `ENTREGADO`, ofreciendo una línea de tiempo interactiva visible para el cliente y actualizable por el administrador.

5.5 Módulo 5: Pasarela de Pagos Digitales Simulada

Simula el comportamiento de una pasarela de pagos omnicanal con soporte para Nequi, Daviplata, PSE, Tarjetas de Crédito y Efectivo contraentrega. Al confirmar la transacción, emite un código de autorización y recibo digital formal, cambiando el estado del pago a `APROBADO` y disparando la preparación inmediata de la orden en cocina.

5.6 Módulo 6: Dashboard Analítico Administrativo

Consola exclusiva para el rol `ADMIN` que totaliza en tiempo real los ingresos brutos en pesos colombianos, pedidos activos en preparación, pedidos entregados, número de especialidades en carta y clientes registrados.

5.7 Matriz de Endpoints RESTful Integrados

Módulo	Método	Endpoint URI	Seguridad	Propósito Funcional
Salud	GET	`/health`	Público	Healthcheck general del microservicio y estado de MongoDB.
Metadatos	GET	`/api/health`	Público	Metadatos institucionales SENA (Aprendiz Karen Yuliana Campo).
Auth	POST	`/api/auth/login`	Público	Inicio de sesión de usuarios y expedición de JWT Bearer.
Auth	GET	`/api/auth/profile`	JWT Bearer	Consulta de perfil del usuario a partir de los claims del token.
Carta	GET	`/api/pizzas`	Público	Catálogo completo de pizzas con filtros por categoría y búsqueda.
Cupones	POST	`/api/cupones/validar`	Público	Validación de cupón promocional y cálculo de descuento en pesos.
Pedidos	POST	`/api/pedidos`	JWT Bearer	Creación de nueva orden con cálculo de subtotales y envío.
Tracking	GET	`/api/pedidos/:numero`	JWT Bearer	Trazabilidad del estado operativo del pedido en 5 fases.
Pagos	POST	`/api/pagos/procesar`	Público	Procesamiento bancario simulado y emisión de comprobante de pago.
Admin	GET	`/api/admin/dashboard`	JWT (ADMIN)	Consola de métricas financieras y operativas del negocio.

6. Configuración de Ambientes y Despliegue con Docker

6.1 Orquestación de Microservicios con Docker Compose

Para garantizar que la integración de módulos funcione de manera idéntica en ambientes de desarrollo, pruebas y producción, se implementó una infraestructura orquestada con **Docker Compose** sobre una red puente aislada denominada `sena-network`:

```
version: '3.8'

networks:
  sena-network:
    driver: bridge

services:
  # Base de Datos Principal NoSQL
  sena-mongodb:
    image: mongo:6-jammy
    container_name: sena-mongodb
    ports:
      - "27017:27017"
    volumes:
      - sena_mongo_data:/data/db
      - ./docker/mongo/init-mongo.js:/docker-entrypoint-initdb.d/init-mongo.js:ro
    networks:
      - sena-network

  # Microservicio Principal en Go Fiber v2
  sena-backend-go:
    image: golang:1.22-alpine
    container_name: sena-backend-go
    working_dir: /app
    ports:
      - "8085:8085"
    environment:
      - PORT=8085
      - MONGO_URI=mongodb://sena-mongodb:27017
      - MONGO_DB=pizzeria_db
      - JWT_SECRET=PizzeriaBellaItaliaPopayan2026SecretKeySecure
    command: ./server
    depends_on:
      - sena-mongodb
    networks:
      - sena-network

  # Frontend Web SPA en React 18 con Vite
  sena-frontend-vite:
    image: node:20-alpine
    container_name: sena-frontend-vite
    ports:
      - "5173:5173"
```

```
    networks:
      - sena-network

  # Servidor de Desarrollo Móvil React Native (Metro Bundler)
  sena-rn-dev:
    image: node:22-alpine
    container_name: sena-rn-dev
    ports:
      - "8081:8081"
    networks:
      - sena-network
```

6.2 Estado Operativo de los Contenedores

Nombre del Contenedor	Imagen Base	Puerto Host → Contenedor	Estado	Rol en la Arquitectura
`sena-mongodb`	mongo:6-jammy	27017 → 27017	UP (Healthy)	Base de datos NoSQL con motor WiredTiger.
`sena-backend-go`	golang:1.22-alpine	8085 → 8085	UP (Activo)	Backend RESTful principal en Go Fiber v2.
`sena-frontend-vite`	node:20-alpine	5173 → 5173	UP (Activo)	Servidor SPA React 18 con Vite.
`sena-rn-dev`	node:22-alpine	8081 → 8081	UP (Activo)	Metro Bundler para aplicación React Native.
`sena-auth-service`	node:20-alpine	8083 → 8083	UP (Activo)	Microservicio de autenticación alternativo.
`sena-pizzeria-db`	mariadb:10.5	3306 → 3306	UP (Healthy)	Base de datos relacional para auditoría.

7. Control de Calidad, Pruebas Unitarias y de Integración

7.1 Pruebas de Integración con Postman y Newman CLI

Para validar de forma automatizada la interoperabilidad de extremo a extremo entre los servicios web del proyecto, se diseñó una suite de pruebas en **Postman v2.1**, ejecutada de manera continua con el runner de integración **Newman**:

> **Resultado de la Ejecución Automatizada:**
> Peticiones ejecutadas: **10 de 10** (100%) | Aserciones evaluadas: **20 de 20** (100% PASS) | Fallos: **0**.
> Tiempo total de ejecución: **463 ms** | Promedio de respuesta: **19 ms**.

Nº	Nombre del Test / Escenario	Método	Aserciones Verificadas	Resultado
01	Healthcheck General	GET /health	Status 200 OK; Microservicio UP; MongoDB Conectado.	**PASS (2/2)**
02	Metadatos Institucionales SENA	GET /api/health	Status 200 OK; Aprendiz Karen Yuliana Campo; Ficha 3186681.	**PASS (2/2)**
03	Login Cliente y Captura JWT	POST /api/auth/login	Status 200 OK; Token JWT válido extraído y almacenado dinámicamente.	**PASS (2/2)**
04	Perfil Autenticado (Bearer)	GET /api/auth/profile	Status 200 OK; Claims descifrados coinciden con cliente autenticado.	**PASS (2/2)**
05	Carta de Pizzas Artesanales	GET /api/pizzas	Status 200 OK; Catálogo retorna array no vacío con precios por tamaño.	**PASS (2/2)**
06	Validación Cupón SENA2026	POST /api/cupones/validar	Status 200 OK; Cupón válido y cálculo exacto del 15% de descuento.	**PASS (2/2)**
07	Creación de Pedido con Descuento	POST /api/pedidos	Status 201 Created; Generación de número de pedido y estado RECIBIDO.	**PASS (2/2)**
08	Procesamiento de Pago Nequi	POST /api/pagos/procesar	Status 200 OK; Transacción APROBADA y referencia bancaria emitida.	**PASS (2/2)**
09	Login Administrador	POST /api/auth/login	Status 200 OK; Verificación de rol ADMIN y captura de JWT administrativo.	**PASS (2/2)**
10	Métricas del Dashboard	GET /api/admin/dashboard	Status 200 OK; Provee métricas de totalVentas y pedidosActivos.	**PASS (2/2)**

7.2 Suite de Pruebas Automatizadas TDD

El repositorio cuenta con un script maestro de integración continua local (`./docker-test.sh`) que ejecuta **180 pruebas unitarias y de integración** organizadas en 28 suites de prueba sobre Node.js Test Runner en un

contenedor Docker aislado. Todas las pruebas se encuentran actualmente en estado **100% GREEN (180 pasadas, 0 fallidas)**.

8. Conclusiones y Enlaces de Acceso

8.1 Conclusiones Técnicas

- **Integración Exitosa de Módulos:** Se logró un acoplamiento perfecto entre la presentación visual (React Web y React Native), los servicios RESTful de backend (Go Fiber v2) y la base de datos NoSQL (MongoDB), permitiendo el flujo continuo de datos a través de todas las capas del sistema.
- **Correspondencia Bidireccional Diseño-Código:** La arquitectura construida es un fiel reflejo de los modelos UML 2.5 diseñados (Diagrama de Clases, Diagrama de Paquetes y Diagrama de Componentes), cumpliendo con los estándares de ingeniería de software exigidos por el SENA.
- **Seguridad Robusta y Eficiente:** La adopción de JWT Bearer con control de acceso basado en roles (RBAC) y cifrado Bcrypt garantiza la protección integral de los datos de los clientes y la confidencialidad de las operaciones comerciales de la pizzería.
- **Resiliencia y Rendimiento:** Gracias al uso del lenguaje Go y su framework Fiber v2, los endpoints responden con tiempos promedio inferiores a 20 ms, optimizando el uso de memoria RAM y CPU en ambientes de producción.

8.2 Enlaces Oficiales de Acceso

Recurso	Tipo	Enlace Web / Referencia
Repositorio Git Oficial	Control de Versiones	github.com/kyulianacampogutierrez-ux/Evidencias.git
Carpeta de la Evidencia en GitHub	Código Fuente y Artefactos	Ver Evidencia en GitHub
Diagrama de Clases UML (Draw.io)	Visor Interactivo	Abrir Diagrama de Clases en app.diagrams.net
Diagrama de Paquetes UML (Draw.io)	Visor Interactivo	Abrir Diagrama de Paquetes en app.diagrams.net
Diagrama de Componentes UML (Draw.io)	Visor Interactivo	Abrir Diagrama de Componentes en app.diagrams.net

KAREN YULIANA CAMPO

Aprendiz ADSO — Ficha: 3186681

Centro de Teleinformática y Producción Industrial (CTPI Popayán) — Regional Cauca

Servicio Nacional de Aprendizaje (SENA) — 2026